

```
GNU nano 8.7.1 week1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, (char *)NULL);

        // This executes only if exec fails
        perror("exec failed");
        exit(1);
    }

    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}

vootla_hemanth@DESKTOP-B177MELO:~$ ./week1
Enter a Linux command: ls

Parent Process
Parent PID: 882
Child PID: 883

Child Process
Child PID: 883
Parent PID: 882
GNU          arithmetic1  class1.c  file.c      getpid      process2    process3.c  week1      week3-waitpid  week3_process  week4_wait.c  week4_zombie.c
arithmetic  arithmetic1.c  dir1      forkprocess  getpid.c    process2.c  shell       week1.c      week3-waitpid.c  week3_process.c  week4_waitpid  week4_zombie
arithmetic.c  class1      file      forkprocess.c  getpid2.c   process3    shell.c     week3       week3.c         week4_wait      week4_waitpid.c
Child process completed.
vootla_hemanth@DESKTOP-B177MELO:~$ |
```